

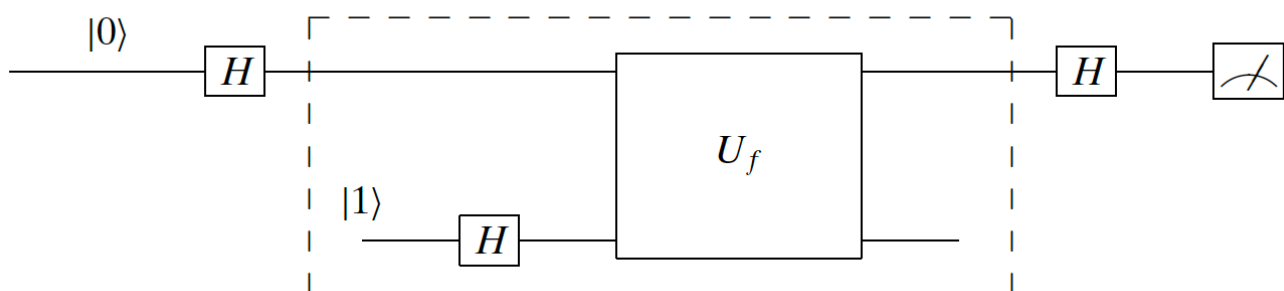
Concurrency and interaction 2024/2025

## Lecture 3 - Deutsch Algorithm

▼ Contents

1. [Deutsch's Algorithm](#)
2. [Deutsch-Jozsa Algorithm](#)

| Constant |                                      | Balanced |                                      |
|----------|--------------------------------------|----------|--------------------------------------|
| (1)      | $f(0) \mapsto 0$<br>$f(1) \mapsto 0$ | (3)      | $f(0) \mapsto 0$<br>$f(1) \mapsto 1$ |
| (2)      | $f(0) \mapsto 1$<br>$f(1) \mapsto 1$ | (4)      | $f(0) \mapsto 1$<br>$f(1) \mapsto 0$ |



▼ 2. Deutsch's Algorithm

Let's consider a Boolean function  $f : \{0, 1\} \rightarrow \{0, 1\}$  that can either be balanced or constant.

- **Constant:** its output is always 0 or always 1
- **Balanced:** outputs 0 for one input and 1 for the other

$U_f$  is our function, usually seen as an oracle, or a black-box function that we want to find if it's balanced or constant

The result for the measurement of the first qubit will be:

- $P(0) = 1$  if the function is **constant**
- $P(1) = 1$  if the function is **balanced**

In general for  $n$ -bit Boolean functions  $f : \{0, 1\}^n \rightarrow \{0, 1\}$ , the generalized Deutsch-Jozsa algorithm can determine if the function is constant or balanced with  $\mathcal{O}(1)$  queries to the function using superposition of inputs + interference.

The classical algorithm needs  $\mathcal{O}(2^{n-1} + 1)$  queries to determine if the function is balanced or constant.

✓ Implement every function and check if it's balanced or constant

```
!pip install pennylane
import pennylane as qml
```

 [Mostrar saída oculta](#)

```
def function(id=1):
    """Defines the oracle function based on ID"""
    if id == 0:
        # Constant function (always 0)
        def oracle():
            pass # Identity - leaves |x>|0> unchanged

    elif id == 1:
        # Constant function (always 1)
        def oracle():
            qml.PauliX(wires=1) # Flips output qubit

    elif id == 2:
        # Balanced function (f(0)=0, f(1)=1)
        def oracle():
            qml.CNOT(wires=[0, 1]) # |x>|0> -> |x>|x>

    elif id == 3:
        # Balanced function (f(0)=1, f(1)=0)
        def oracle():
            qml.PauliX(wires=0)
            qml.CNOT(wires=[0, 1])
            qml.PauliX(wires=0)

    return oracle

def Deutsch(id=1):
    # Create quantum device with 2 qubits
    dev = qml.device("default.qubit", wires=2, shots=1)

    @qml.qnode(dev)
    def circuit():
        # Prepare input qubit in |1> for phase kickback
        qml.PauliX(wires=0)

        # Apply Hadamard to both qubits (superposition)
        qml.Hadamard(wires=0)
        qml.Hadamard(wires=1)

        # Apply the oracle function
        function(id)()

        # Apply Hadamard to input qubit (interference)
        qml.Hadamard(wires=0)

        # Measure first qubit
        return qml.sample(qml.PauliZ(0))

    # Execute the circuit
    result = circuit()

    # Convert measurement (-1/+1) to (1/0)
    measurement = 0 if result == 1 else 1

    # Determine function type
    if measurement == 0:
        print(f"Function {id} is CONSTANT")
    else:
        print(f"Function {id} is BALANCED")

    return measurement

print("Deutsch-Jozsa Algorithm Execution:")
print("-----")
for fid in range(4):
    print(f"\nTesting function {fid}:")
    Deutsch(fid)
```

 Deutsch-Jozsa Algorithm Execution:

-----  
Testing function 0:

```

Function 0 is BALANCED

Testing function 1:
Function 1 is BALANCED

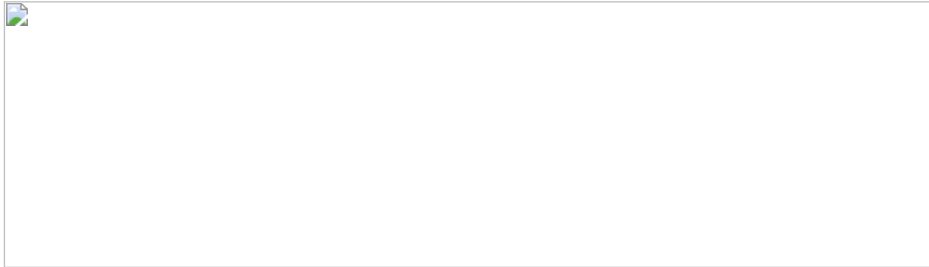
Testing function 2:
Function 2 is BALANCED

Testing function 3:
Function 3 is BALANCED

```

### ✓ 3. Deutsch-Jozsa Algorithm

Deutsch-Jozsa algorithm is the generalization of the previous algorithm for boolean functions for  $n$  bit strings rather than single bit string inputs.



The results for the single input case also remain true in this setting with minor differences:

- Hadamard uniform superposition over all qubits instead of a single qubit
- We will measure all input qubits instead of the single qubit
- $P(000\dots 0) = 1$  if the function is **constant**
- $P(000\dots 0) = 0$  if the function is **balanced**

### ✓ Implement Deutsch-Jozsa Algorithm

```

import pennylane as qml
from pennylane import numpy as np

def function(n_qubits, id=0):
    """Defines the oracle function (constant or balanced)"""
    def oracle():
        if id == 0:
            # Constant function (always 0)
            pass
        elif id == 1:
            # Constant function (always 1)
            qml.PauliX(wires=n_qubits) # Flip ancilla
        else:
            # Balanced function (example: XOR with first qubit)
            for i in range(n_qubits):
                qml.CNOT(wires=[i, n_qubits])
    return oracle

def Deutsch_Jozsa(nqubits_input, nqubits_ancilla=1, id=0):
    """Generalized Deutsch-Jozsa algorithm"""
    n_qubits = nqubits_input + nqubits_ancilla

    # Initialize device
    dev = qml.device("default.qubit", wires=n_qubits, shots=1000)

    @qml.qnode(dev)
    def circuit():
        # Input qubits preparation (|1> for phase kickback)
        for i in range(nqubits_input):
            qml.PauliX(wires=i)

        # Ancilla preparation (|1> for phase kickback)
        for i in range(nqubits_input, n_qubits):
            qml.PauliX(wires=i)

        # Superposition (H on all qubits)
        for i in range(n_qubits):
            qml.Hadamard(wires=i)

        # Apply oracle function
        function(nqubits_input, id)()

```

```

# Interference (H on input qubits)
for i in range(nqubits_input):
    qml.Hadamard(wires=i)

# Measurement (on input qubits)
return [qml.sample(qml.PauliZ(wires=i)) for i in range(nqubits_input)]

# Execute
results = circuit()

# Convert results (-1/+1) to (1/0)
measurements = np.where(np.array(results) == 1, 0, 1)

# Count all-zero measurements
all_zeros = np.sum(np.all(measurements == 0, axis=0))

# Determine if constant (all measurements 0) or balanced
if all_zeros == 1000:
    print(f"Function {id} is CONSTANT")
else:
    print(f"Function {id} is BALANCED")

return measurements

print("Generalized Deutsch-Jozsa Algorithm:")
print("-----")
n = 3 # Number of input qubits
for fid in [0, 1, 2]: # Test constant and balanced cases
    print(f"\nTesting {n}-qubit function {fid}:")
    Deutsch_Jozsa(nqubits_input=n, id=fid)

```

↩ Generalized Deutsch-Jozsa Algorithm:  
-----

Testing 3-qubit function 0:  
Function 0 is BALANCED

Testing 3-qubit function 1:  
Function 1 is BALANCED

Testing 3-qubit function 2:  
Function 2 is CONSTANT

## ✓ Implement the Deutsch-Jozsa algorithm for the 3-bit string function with truth table

| x <sub>0</sub> | x <sub>1</sub> | y |
|----------------|----------------|---|
| 0              | 0              | 0 |
| 0              | 1              | 1 |
| 1              | 0              | 1 |
| 1              | 1              | 0 |

```

def create_oracle(truth_table):
    """Creates a quantum oracle from a 3-bit truth table"""
    def oracle():
        # We'll use an intermediate ancilla qubit for multi-controlled operations
        for input_val, output in enumerate(truth_table):
            if output == 1:
                binary_str = format(input_val, '03b') # 3-bit binary string

                # Apply X gates to qubits where bit is 0
                flip_qubits = [i for i, bit in enumerate(binary_str) if bit == '0']
                for qubit in flip_qubits:
                    qml.PauliX(wires=qubit)

                # Implement 3-controlled NOT using two Toffoli gates
                qml.Toffoli(wires=[0, 1, 4]) # Use extra ancilla qubit (wire 4)
                qml.Toffoli(wires=[2, 4, 3]) # Final control with ancilla
                qml.Toffoli(wires=[0, 1, 4]) # Uncompute first Toffoli

                # Undo the X gates
                for qubit in flip_qubits:
                    qml.PauliX(wires=qubit)
    return oracle

def Deutsch_Jozsa(truth_table):
    """Determines if a 3-bit function is constant or balanced"""
    n_qubits = 5 # 3 input + 1 output ancilla + 1 temporary ancilla
    dev = qml.device("default.qubit", wires=n_qubits, shots=1000)

    @qml.qnode(dev)

```

```

def circuit():
    # Initialize output ancilla in |1> state for phase kickback
    qml.PauliX(wires=3)

    # Apply Hadamard to all qubits
    for i in range(n_qubits):
        qml.Hadamard(wires=i)

    # Apply the oracle
    create_oracle(truth_table)()

    # Apply Hadamard to input qubits (interference)
    for i in range(3):
        qml.Hadamard(wires=i)

    # Measure input qubits
    return [qml.sample(qml.PauliZ(wires=i)) for i in range(3)]

# Execute the circuit
results = circuit()

# Convert results (-1/+1) to (1/0)
measurements = np.where(np.array(results) == 1, 0, 1)

# Check if all measurements are 0
if np.all(measurements == 0):
    return "CONSTANT function"
else:
    return "BALANCED function"

# Example truth tables
constant_zero = [0, 0, 0, 0, 0, 0, 0, 0]
constant_one = [1, 1, 1, 1, 1, 1, 1, 1]
balanced_example1 = [0, 1, 0, 1, 0, 1, 0, 1]
balanced_example2 = [1, 0, 1, 0, 1, 0, 1, 0]

# Test the algorithm
print("Testing constant function (all zeros):", Deutsch_Jozsa(constant_zero))
print("Testing constant function (all ones):", Deutsch_Jozsa(constant_one))
print("Testing balanced function (alternating):", Deutsch_Jozsa(balanced_example1))
print("Testing balanced function (inverse pattern):", Deutsch_Jozsa(balanced_example2))

Testing constant function (all zeros): CONSTANT function
Testing constant function (all ones): CONSTANT function
Testing balanced function (alternating): BALANCED function
Testing balanced function (inverse pattern): BALANCED function

```

Consider the following 3-qubit input state truth table:

| Input | Output |
|-------|--------|
| 000   | 1      |
| 001   | 1      |
| 010   | 1      |
| 011   | 1      |
| 100   | 0      |
| 101   | 0      |
| 110   | 0      |
| 111   | 0      |

implement the algorithm for this function

```

def create_oracle():
    """Creates the oracle for the given truth table"""
    def oracle():
        # Implement the function where first bit determines output
        # f(x) = NOT(x0), since output is 1 when x0=0 and 0 when x0=1

        # We can implement this with a single CNOT from x0 to ancilla
        # But need to account for phase kickback properly

        # Since we want |x>|b> → |x>|b⊕f(x)>
        # And our f(x) = NOT(x0), we can do:
        qml.PauliX(wires=3) # Prepare |1> on ancilla for phase kickback
        qml.CNOT(wires=[0, 3]) # This implements f(x) = x0
        qml.PauliX(wires=3) # Now we have f(x) = NOT(x0)
    return oracle

def Deutsch_Jozsa():
    """Determines if the given 3-bit function is constant or balanced"""

```

```

n_qubits = 4 # 3 input + 1 ancilla
dev = qml.device("default.qubit", wires=n_qubits, shots=1000)

@qml.qnode(dev)
def circuit():
    # Initialize ancilla in |1> state for phase kickback
    qml.PauliX(wires=3)

    # Apply Hadamard to all qubits
    for i in range(n_qubits):
        qml.Hadamard(wires=i)

    # Apply the oracle
    create_oracle()

    # Apply Hadamard to input qubits (interference)
    for i in range(3):
        qml.Hadamard(wires=i)

    # Measure input qubits
    return [qml.sample(qml.PauliZ(wires=i)) for i in range(3)]

# Execute the circuit
results = circuit()

# Convert results (-1/+1) to (1/0)
measurements = np.where(np.array(results) == 1, 0, 1)

# Check if all measurements are 0
if np.all(measurements == 0):
    return "CONSTANT function"
else:
    return "BALANCED function"

# Test the algorithm with the given function
print("Testing the given 3-bit function:")
result = Deutsch_Jozsa()
print("Result:", result)

# Additional verification
print("\nTruth table verification:")
print("Input | Output")
print("-----|-----")
for i in range(8):
    bits = format(i, '03b')
    output = 1 if bits[0] == '0' else 0 # Our function: NOT(first bit)

```

Testing the given 3-bit function:  
Result: BALANCED function

Truth table verification:

Input | Output

| Input | Output |
|-------|--------|
| 000   | 1      |
| 001   | 1      |
| 010   | 1      |
| 011   | 1      |
| 100   | 0      |
| 101   | 0      |
| 110   | 0      |
| 111   | 0      |